



UNINASSAU

Polimorfismo

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

POLIMORFISMO

PILARES DE POO

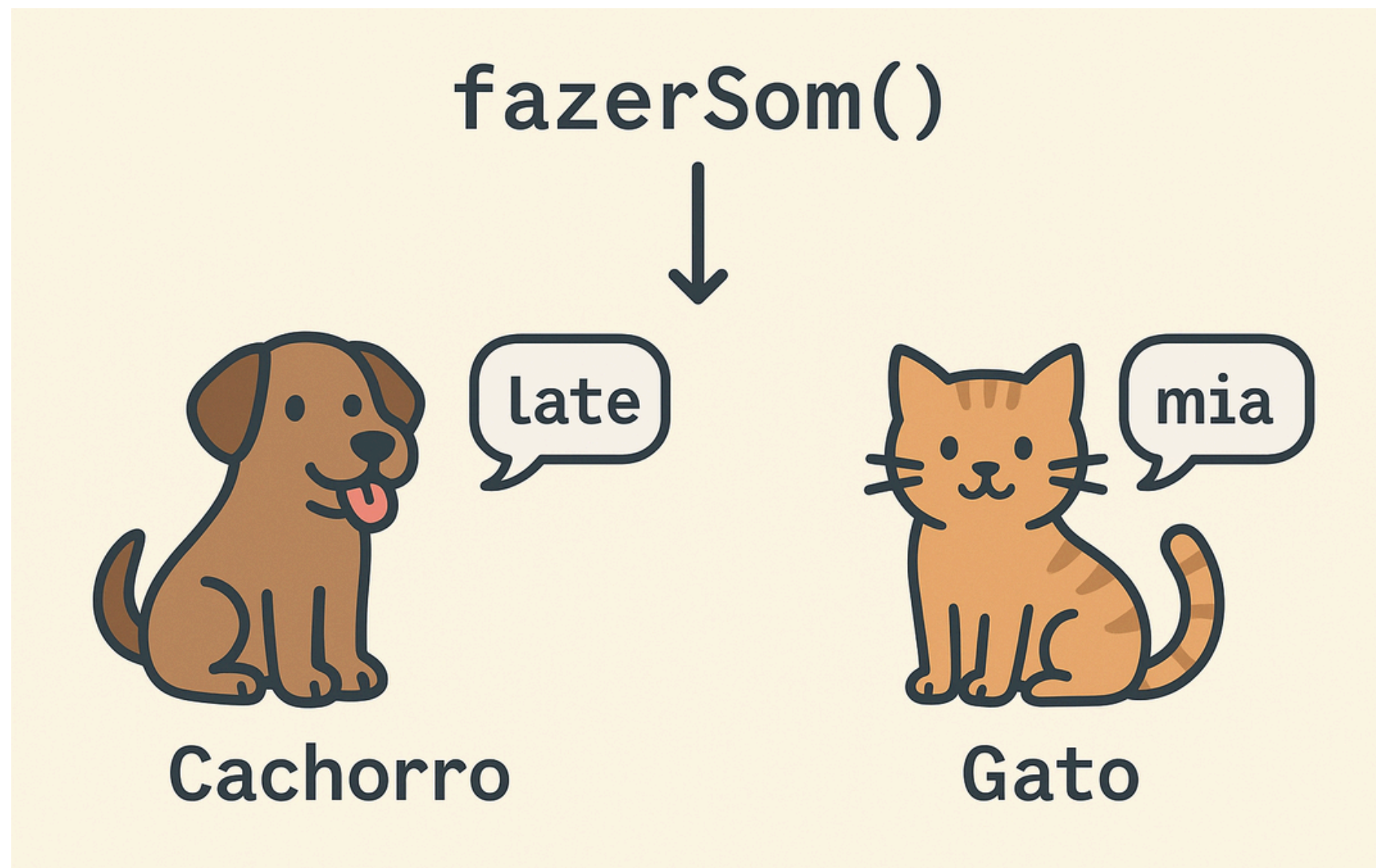


O QUE É POLIMORFISMO?

- Do grego "muitas formas" - a capacidade de um método ou objeto assumir diferentes comportamentos dependendo do contexto.
- Permite que objetos de diferentes classes sejam tratados como objetos de uma classe base comum, mantendo seus comportamentos específicos.

- Exemplo:

Um método "fazerSom()" que age diferentemente para objetos "Cachorro" (late) e "Gato" (mia), mas é chamado da mesma forma.



POR QUE POLIMORFISMO É IMPORTANTE?

- **Flexibilidade Máxima**

O mesmo código pode funcionar com diferentes tipos de objetos, adaptando-se automaticamente aos comportamentos específicos de cada classe sem modificações.

- **Reutilização Inteligente**

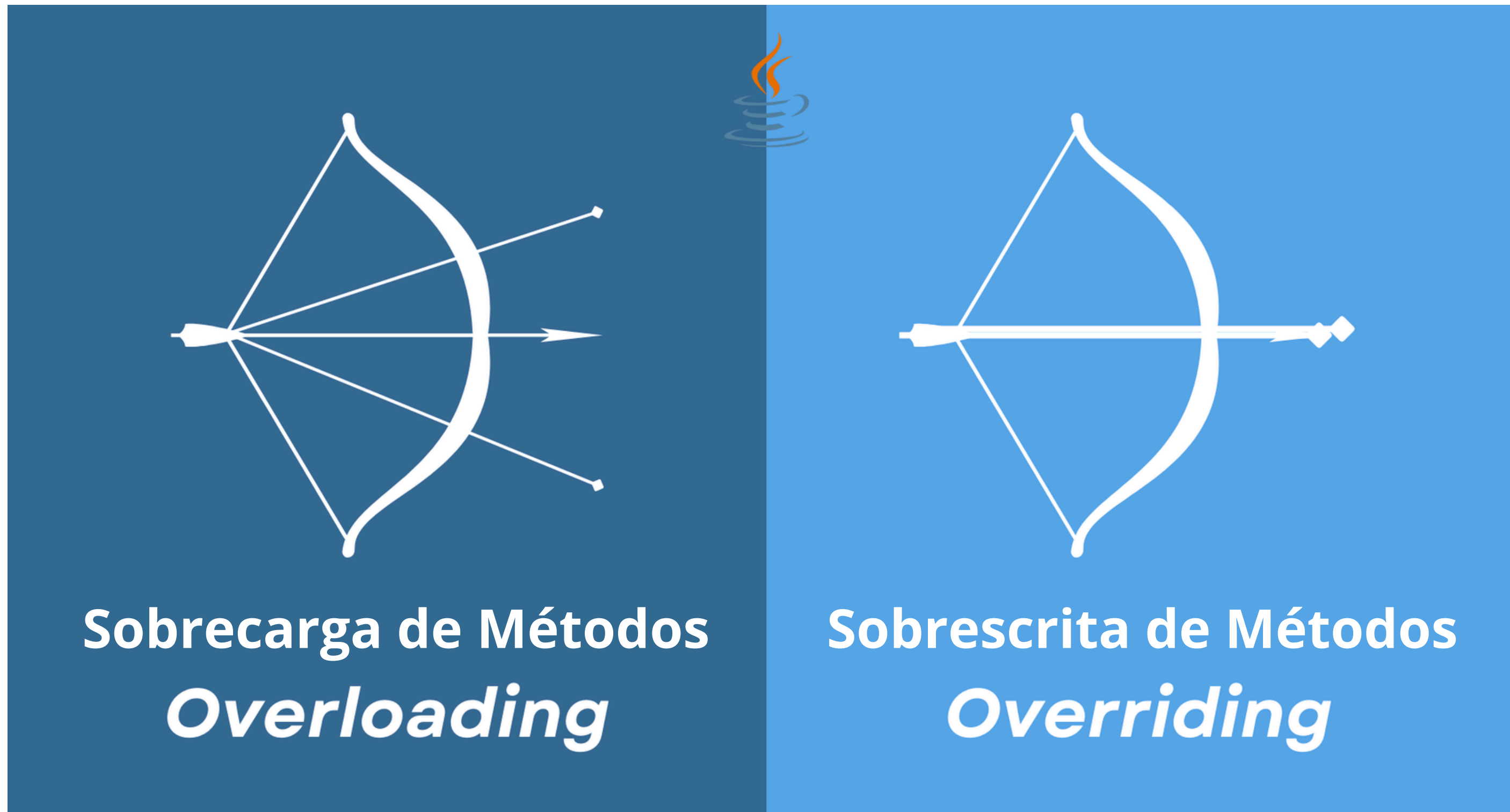
Facilita enormemente a extensão do sistema, permitindo adicionar novas funcionalidades sem precisar alterar o código já existente e testado.

- **Manutenção Simplificada**

Resulta em código mais limpo, modular e intuitivo, reduzindo a complexidade de manutenção e facilitando a colaboração entre desenvolvedores.



TIPOS DE POLIMORFISMO



Mesma classe, métodos com o mesmo nome mas parâmetros diferentes.

Uma subclasse redefine um método da superclasse.

TIPOS DE POLIMORFISMO

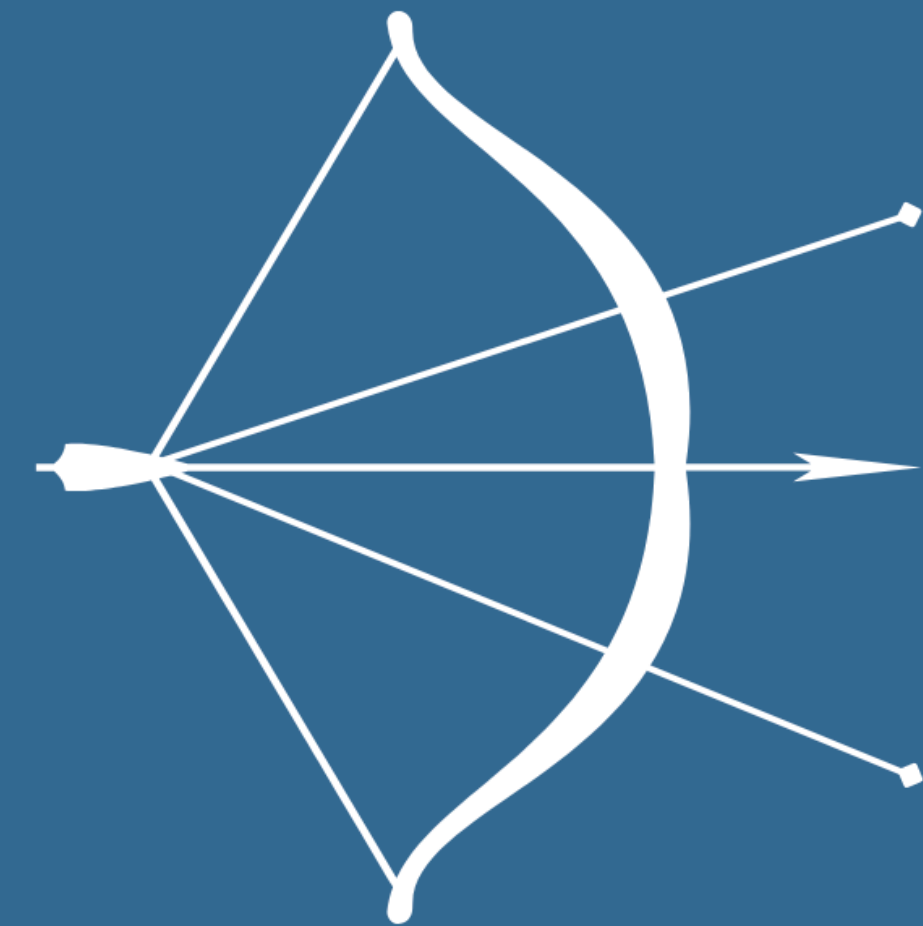
Sobrecarga de Métodos (Overloading)

- Métodos com o mesmo nome, mas assinaturas diferentes. O compilador decide qual versão usar baseado nos parâmetros fornecidos.



EXEMPLO DE SOBRECARGA

```
class Calculadora {  
    int somar(int a, int b) {  
        return a + b;  
    }  
  
    double somar(double a, double b) {  
        return a + b;  
    }  
  
    int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```



Overloading

TIPOS DE POLIMORFISMO

Sobrescrita de Métodos (Overriding)

- Método redefinido nas subclasses, mas chamado através de uma referência da superclasse. A decisão de qual implementação usar acontece durante a execução.



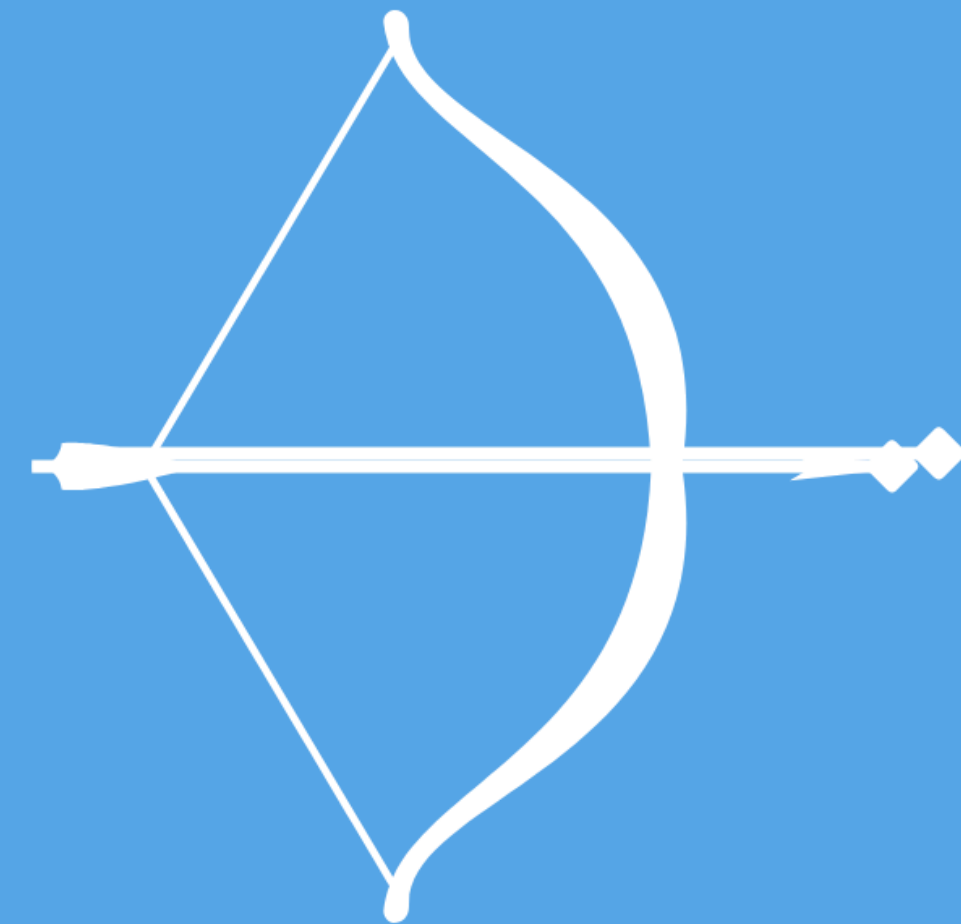
EXEMPLO DE SOBREPOSIÇÃO

```
class Animal {  
    void fazerSom() {  
        System.out.println("Som de animal");  
    }  
}  
  
class Cachorro extends Animal {  
    @Override  
    void fazerSom() {  
        System.out.println("Au au!");  
    }  
}  
  
class Gato extends Animal {  
    @Override  
    void fazerSom() {  
        System.out.println("Miau!");  
    }  
}
```



EXEMPLO DE SOBREPOSIÇÃO

```
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Cachorro();  
        Animal a2 = new Gato();  
  
        a1.fazerSom(); // Au au!  
        a2.fazerSom(); // Miau!  
    }  
}
```

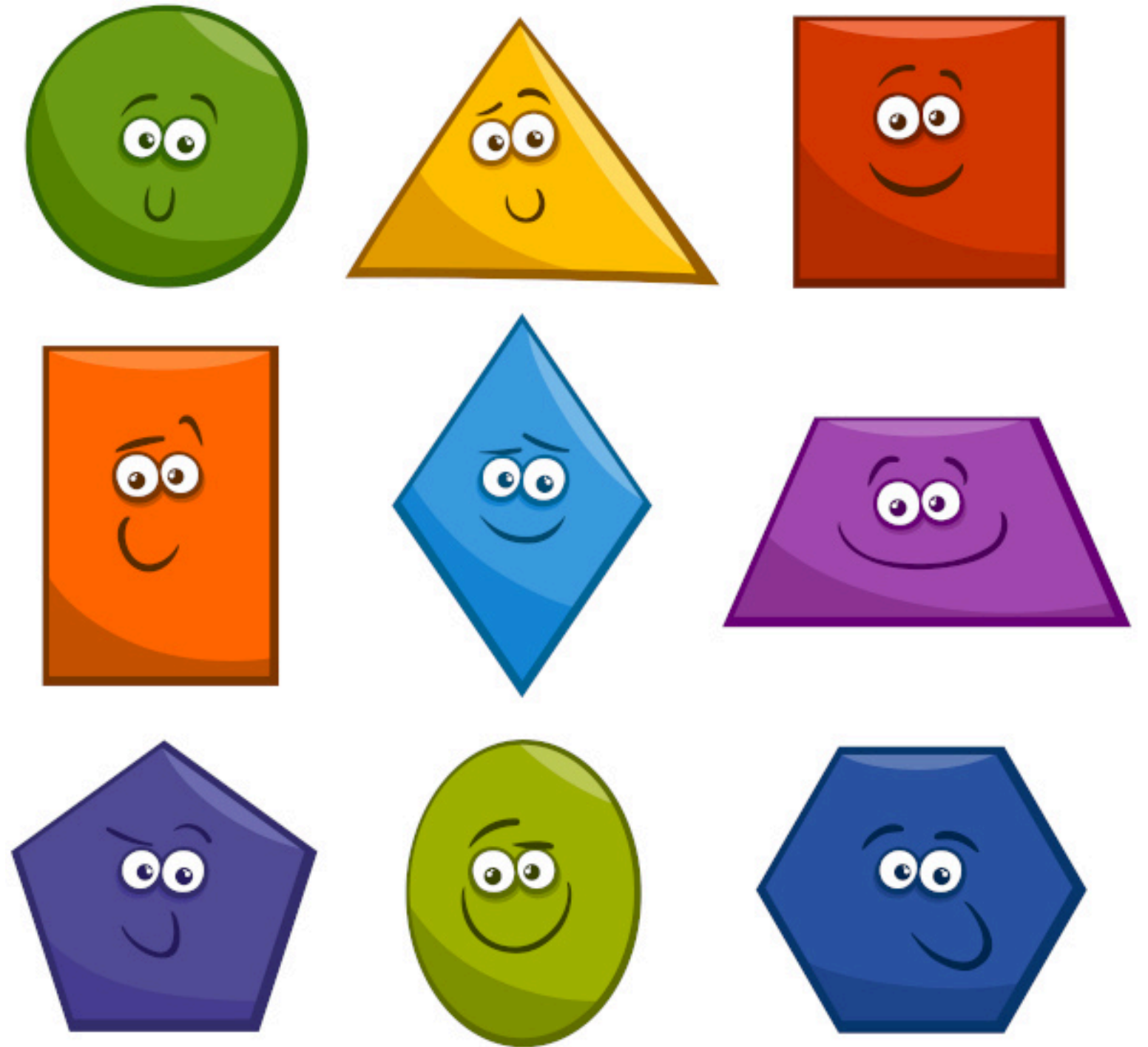


Overriding

EXEMPLOS

EXEMPLO 1: CLASSE FORMA

- Crie uma classe **Forma** com método **calcularArea()**.
- Implemente subclasses **Circulo**, **Quadrado** e **Triangulo**.
- No **main**, crie um **array de Forma** e percorra chamando **calcularArea()**.



EXEMPLO 1: CLASSE FORMA

Passo 1:

- Crie uma classe **Forma** com método **calcularArea()**.
- Implemente subclasse **Circulo**.

```
// Classe base
abstract class Forma {
    abstract double calcularArea();
}

// Subclasse Círculo
class Circulo extends Forma {
    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }

    @Override
    double calcularArea() {
        return Math.PI * raio * raio;
    }
}
```

EXEMPLO 1: CLASSE FORMA

Passo 2:

- Implemente subclasse **Quadrado** e **Triangulo**.

```
// Subclasse Quadrado
class Quadrado extends Forma {
    private double lado;

    public Quadrado(double lado) {
        this.lado = lado;
    }

    @Override
    double calcularArea() {
        return lado * lado;
    }
}
```

```
// Subclasse Triângulo
class Triangulo extends Forma {
    private double base, altura;

    public Triangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    double calcularArea() {
        return (base * altura) / 2;
    }
}
```

EXEMPLO 1: CLASSE FORMA

Passo 3:

- No **main**, crie um **array de Forma** e percorra chamando **calcularArea()**.

```
// Programa principal
public class Main {
    public static void main(String[] args) {
        Forma[] formas = new Forma[3];
        formas[0] = new Circulo(3);
        formas[1] = new Quadrado(4);
        formas[2] = new Triangulo(6, 2);

        for (Forma f : formas) {
            System.out.println("Área: " + f.calcularArea());
        }
    }
}
```

Saída esperada:

```
Área: 28.274333882308138
Área: 16.0
Área: 6.0
```


EXEMPLO 2: CLASSE FUNCIONARIO

- Implemente **Funcionario** com método **calcularSalario()**.
- Crie subclasses **Gerente** e **Programador**, cada uma calculando de forma diferente.
- No **main**, use polimorfismo para chamar o mesmo método.



EXEMPLO 2: CLASSE FUNCIONARIO

Passo 1:

- Implemente **Funcionario** com método **calcularSalario()**.

```
// Classe base  
abstract class Funcionario {  
    protected String nome;  
  
    public Funcionario(String nome) {  
        this.nome = nome;  
    }  
  
    abstract double calcularSalario();  
}
```

EXEMPLO 2: CLASSE FUNCIONARIO

Passo 2:

- Crie subclasses **Gerente** e **Programador**, cada uma calculando de forma diferente.

```
// Subclasse Gerente
class Gerente extends Funcionario {
    private double salarioBase;
    private double bonus;

    public Gerente(String nome, double salarioBase, double bonus) {
        super(nome);
        this.salarioBase = salarioBase;
        this.bonus = bonus;
    }

    @Override
    double calcularSalario() {
        return salarioBase + bonus;
    }
}
```

EXEMPLO 2: CLASSE FUNCIONARIO

Passo 2:

- Crie subclasses **Gerente** e **Programador**, cada uma calculando de forma diferente.

```
// Subclasse Programador
class Programador extends Funcionario {
    private double salarioBase;
    private int horasExtras;
    private double valorHoraExtra;

    public Programador(String nome, double salarioBase, int horasExtras, double valorHoraExtra) {
        super(nome);
        this.salarioBase = salarioBase;
        this.horasExtras = horasExtras;
        this.valorHoraExtra = valorHoraExtra;
    }

    @Override
    double calcularSalario() {
        return salarioBase + (horasExtras * valorHoraExtra);
    }
}
```

EXEMPLO 2: CLASSE FUNCIONARIO

Passo 3:

- No **main**, use polimorfismo para chamar o mesmo método.

```
// Programa principal
public class Main2 {
    public static void main(String[] args) {
        Funcionario f1 = new Gerente("Alice", 5000, 2000);
        Funcionario f2 = new Programador("Bob", 4000, 10, 50);

        System.out.println(f1.nome + " - Salário: " + f1.calcularSalario());
        System.out.println(f2.nome + " - Salário: " + f2.calcularSalario());
    }
}
```

Saída esperada:

```
Alice - Salário: 7000.0
Bob - Salário: 4500.0
```


Exercícios Propostos

EXERCÍCIO 1: MEIOS DE TRANSPORTE

- Crie uma interface **MeioTransporte** com o método **mover()**.
- Implemente **Carro**, **Bicicleta** e **Avião**.
- Use polimorfismo para percorrer uma lista de transportes chamando **mover()**.



EXERCÍCIO 2: INSTRUMENTOS MUSICAIS

- Crie uma classe abstrata **Instrumento** com o método **tocar()**.
- Implemente as subclasses **Violao**, **Piano** e **Bateria**, cada uma imprimindo sons diferentes no método **tocar()**.
- No **main**, crie uma lista de **Instrumento** e percorra tocando cada um.



EXERCÍCIO 3: SISTEMA DE PAGAMENTOS

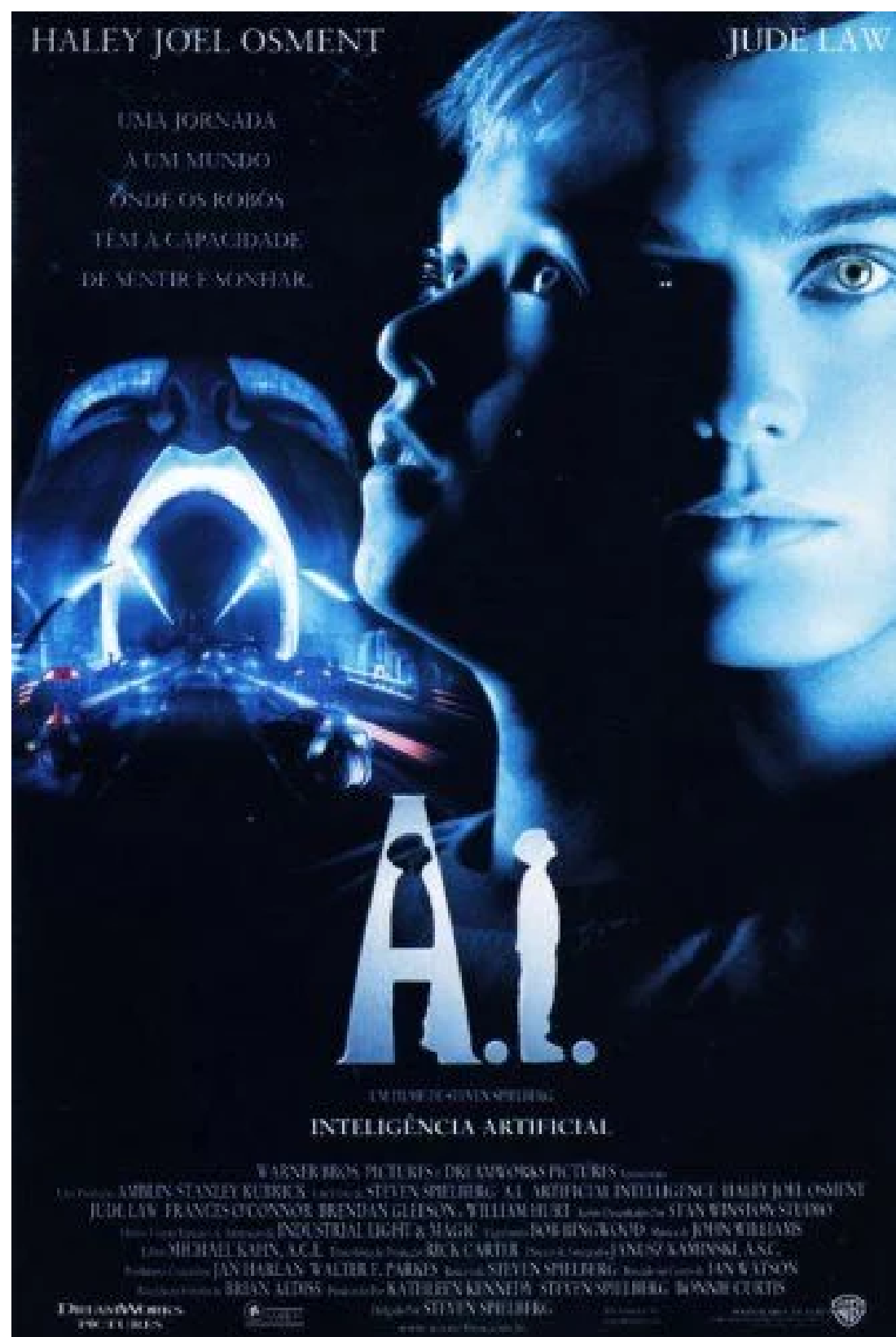
- Implemente uma interface **Pagamento** com o método **processarPagamento(double valor)**.
- Crie as classes **CartaoCredito**, **Boleto** e **Pix**, cada uma implementando a interface de forma diferente (mensagens personalizadas).
- No **main**, use polimorfismo para percorrer uma lista de métodos de pagamento e processar diferentes valores.



REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

A.I. - Inteligência Artificial (2001)



Na metade do século XXI, o efeito estufa derreteu uma grande parte das colatas polares da Terra, fazendo com que boa parte das cidades litorâneas do planeta fiquem parcialmente submersas. Para controlar este desastre ambiental a humanidade conta com o auxílio de uma nova forma de computador independente, com inteligência artificial, conhecido como A.I. É neste contexto que vive o garoto David Swinton (Haley Joel Osment), que irá passar por uma jornada emocional inesquecível.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code